Séquence 1

EXERCICES

Exercice 1: quelques pièges de l'affectation

On définit la fonction `calcul_v1` suivante:

```
1  def calcul_v1(n):
2      resultat = 0
3      for i in range(n):
4          resultat = resultat + 1
5      return resultat
```

1. Donner deux constructions élémentaires différentes ainsi que le numéro de la ligne où elles apparaissent.
2. On modifie légèrement la fonction précédente, pour obtenir une autre version:

```
1  def calcul_v2(n):
2      for i in range(n):
3          resultat = resultat + 1
4      return resultat
```

L'exécution de `calcul_v2(0)` provoque cette fois-ci l'erreur suivante:

```
1  ---------------------------------------------------------------------------
2  UnboundLocalError                         Traceback (most recent call last)
3  <ipython-input-5-e000315c4ef2> in <module>
4  ----> 1 calcul_v2(0)
5
6  <ipython-input-4-012f1c01c96b> in calcul_v2(n)
7        2     for i in range(n):
8        3         resultat = resultat + 1
9  ----> 4     return resultat
10
11 UnboundLocalError: local variable 'resultat' referenced before assignment
```

En comparant avec la première version, donner l'origine cette erreur. Proposer une explication de la situation.

3. Un élève exécute la séquence d'instructions suivante:

```
1  calcul_v1(100)
2  somme_100 = resultat
```

Malheureusement cette séquence échoue. L'interpréteur python affiche l'erreur

```
NameError: name 'resultat' is not defined
```

Cette erreur semble curieuse à priori car la variable `resultat` a été déclarée et affectée dans la fonction `calcul_v1`.

Faire une hypothèse sur la *visibilité* d'une variable déclarée à l'intérieur d'une fonction.

Exercice 2: des étoiles en boucle

Préambule: compléments sur les chaînes de caractère

Utilisation des opérateurs `+` *et* `*`

```
ch = '_/\_'
print(ch)
```

```
_/\_
```

L'opérateur `+` permet de concaténer deux chaînes.

```
print(ch + ch + ch)
```

```
_/\__/\__/\_
```

On peut aussi utiliser l'opérateur `*` entre une chaîne et un entier.

```
ch = ch * 3
print(ch)
```

```
_/\__/\__/\_
```

Dans ce dernier exemple `ch` référence un nouvel objet résultant de la concaténation `_/\_` + `_/\_` + `_/\_`.

Le caractère d'échappement `\`

Sa présence dans une chaîne modifie le caractère qui le suit. Ainsi `\n` ne désigne pas `n` mais un saut de ligne.

```
print('100\n200\n300')
```

```
100
200
300
```

Enoncé

On considère la fonction `etoilef` suivante, dont la documentation est incomplète.

```python
def etoilef(n):
    """
    A compléter
    """
    a = ''
    for i in range(1, n+1):
        a = a + '*' * i + '\n'
    return a
```

1. Que réalise cette fonction? En cas de difficulté, on peut réaliser quelques appels `print(etoilef(n))`, `n` étant un entier naturel supérieur ou égal à 1.
2. Compléter la documentation de la fonction.
3. Sur la i-ème ligne, i étant un entier supérieur à 1, combien a-t-on d'étoiles ?
4. Transformer la fonction `etoilef` en une fonction `etoilew` qui utilise une boucle conditionnelle (`while`) et qui répond aux mêmes spécifications.

Exercice 3: diversité des langages

Aucune connaissance des langages cités ci-dessous n'est nécessaire pour résoudre l'exercice. Il s'agit d'un travail sur les similitudes et les différences existant entre ces langages.

Le mélange de Fisher-Yates est un algorithme qui permet de mélanger des objets (voir sa description sur Wikipedia). On se propose de comparer la traduction cet algorithme dans divers langages de programmation.

`Pascal (1970)`

```pascal
procedure shuffleList(var tab: tableau);
var
    i, j, tmp : integer;
begin
    for i := 1 to High(tab) - Low(tab) do begin
        j := random(i + 1);
        if j < i then
            tmp := tab[i + Low(tab)];
            tab[i + Low(tab)] := tab[j + Low(tab)];
            tab[j + Low(tab)] := tmp
        end;
    end;
end;
```

`C (1972)`

```
void shuffle(int tab[], int n) {
    int i;
    for (i = 1; i < n; ++i) {
        int j = random(i + 1);
        if (j < i) {
            int tmp = tab[i];
            tab[i] = tab[j];
            tab[j] = tmp;
        }
    }
}
```

`Go (2009)`

```
func shuffle(tab []int) {
    for i := 1; i < len(tab); i++ {
        j := rand.Intn(i + 1)
        if j < i {
            tmp := tab[i]
            tab[i] := tab[j]
            tab[j] := tmp
        }
    }
}
```

`ECMAScript (Javascript) ES 6 (2015)`

```
const shuffle = (tab) => {
for (let i = 1; i < tab.length; i++) {
    let j = Math.floor((i + 1) * Math.random())
    if (j < i) {
        let temp = tab[j]
        tab[j] = tab[i]
        tab[i] = temp
        }
    }
}
```

`Python (1991)`

```
def shuffle(tab):
    for i in range(1, len(tab)):
        j = random.randint(0, i)
        if j < i:
            tab[i], tab[j] = tab[j], tab[i]
```

1. Comment sont délimités les blocs d'instructions ?
2. Est-il nécessaire de déclarer les variables *locales* dans une fonction ?
3. Comment est construite une boucle `for` ?
4. A partir des informations proposées, de quel(s) langage(s), le `Go` est-il proche ?

Réponses

	Pascal	C	Go	Python	Javascript (ES 6)
Délimiteurs					
Variables locales					
Structure boucle for					

Exercice 4: spécifier une fonction et tests

Lucas a déjà eu 3 notes sur 20 en NSI:

- 14 coef. 1
- 15 coef. 2
- 16 coef. 2

La dernière évaluation prévue sera affectée d'un coefficient 5. Lucas veut prévoir sa moyenne trimestrielle en écrivant une fonction `moyenne` qui prendra en paramètre sa dernière note et qui renvoie sa moyenne trimestrielle.

1. Ecrire les spécifications de la fonction `moyenne`.
2. Donner une assertion qui permet de garantir la précondition.
3. Lucas a bien pris note des bonnes pratiques de programmation. Il commence par écrire une fonction de tests de sa fonction `moyenne`:

   ```
   def test_moyenne():
       # Cas limites
       assert moyenne(0) == ...
       assert moyenne(20) == ...
       # Cas typique
       assert moyenne(10) == ...
       return 'Tests OK'
   ```

   Compléter cette fonction de test.
4. Ecrire la fonction `moyenne` et tester la.

Exercice 5: divisibilité

Un moyen simple pour savoir si un entier naturel a est divisible par un entier naturel k non nul, est de tester le reste de la division de a par k. S'il est nul alors a est divisible par k. La fonction `est_divisible` ci-dessous code

cette propriété.

```
1  def est_divisible(a, k):
2      """
3      Renvoie vrai (True) si a est divisible par k, faux (False) sinon.
4      a: entier naturel
5      k: entier naturel strictement positif
6      """
7      assert isinstance(a, int), "Erreur, a entier"
8      assert a >= 0, "Erreur, a entier naturel"
9      assert isinstance(k, int), "Erreur, k entier"
10     assert k > 0, "Erreur, k strictement positif"
11
12     return a % k == 0
13
14 def test_divisible():
15     assert est_divisible(0, 1) is True
16     assert est_divisible(2, 1) is True
17     assert est_divisible(10, 3) is False
18     return 'Tests OK'
```

1. Combien de paramètres requiert la fonction `est_divisible` ?
2. Quel est l'intérêt des lignes 7 à 10 ?
3. Tester la fonction `est_divisible`.
4. Prévoir le résultat renvoyé par l'appel `test_divisible()` si on rajoute avant la ligne `return ...` l'instruction `assert est_divisible(10, 0) is False`.

Exercice 6: année bissextile

Dans le calendrier grégorien, une année est bissextile si (voir article wikipedia) on se trouve dans un des cas suivants:

- l'année est divisible par 4 et non divisible par 100 ;
- l'année est divisible par 400

On souhaite coder une fonction `est_bissextile` qui prend en paramètre une année et qui renvoie le résultat logique vrai ou faux (`True` ou `False` en python) selon que l'année est bissextile ou non. Pour simplifier, on considère que le passage au calendrier grégorien a eu lieu en 1583.

1. Compléter les spécifications de la fonction `est_bissextile` ci-dessous. Seuls les `...` sont à compléter. Ne pas supprimer l'instruction `pass` qui sera remplacée par votre code plus tard.

```python
# A compléter

def est_bissextile(...):
    """
    ...
    """
    pass
```

2. Ecrire une fonction de tests `test_bissextile` en utilisant les résultats disponibles sur la page wikipedia.

```python
# A compléter après avoir supprimer l'instruction pass

def test_bissextile():
    pass
```

3. Compléter la fonction `est_bissextile` en remplaçant l'instruction `pass` par votre code et en ré-utilisant la fonction `est_divisble` de l'exercice précédent.
4. Tester votre fonction.